```
import os
import hmac
import json
import time
import hashlib
import asyncio
from datetime import datetime, timezone,
timedelta
from urllib.parse import parse_qsl, quote
import aiohttp
from aiohttp import web
from aiogram import Bot, Dispatcher, F
from aiogram.filters import CommandStart,
Command
from aiogram.types import (
    Message, CallbackQuery, FSInputFile,
    InlineKeyboardMarkup,
InlineKeyboardButton,
    ReplyKeyboardMarkup, KeyboardButton,
ChatMemberUpdated,
    WebAppInfo, MenuButtonWebApp,
)

TOKEN =
"".join(os.environ["BOT_TOKEN"].split())

SITE_URL = "https://demirbek411-
ops.github.io/TalabaMed-site/"
SITE_ORIGIN = "https://demirbek411-
ops.github.io"
```

```
SUPABASE_URL =
os.environ.get("SUPABASE_URL",
"").strip().rstrip("/")
SUPABASE_KEY =
"".join(os.environ.get("SUPABASE_KEY",
"").split())
ADMIN_ID = int(os.environ.get("ADMIN_ID",
"0") or 0)
TZ = timezone(timedelta(hours=5))  #
Toshkent vaqti

SCHEDULE_URL = "https://demirbek411-
ops.github.io/Dars-jadvali-/"

CONTACT = "@Demirbek_17_09"

WELCOME_TEXT = (
    "🩺 TALABA MED: tibbiyot talabasi uchun
test ilovasi\n\n"
    "Imtihon oldidan faqat darslik emas,
o'zingizni sinab ko'rish ham kerak. 📲\n\n"
    "🔹 O'zbek va rus potoklari uchun
yakuniy testlar ham bor: Gistologiya,
Anatomiya, Psixologiya (3000 dan ortiq
test)\n"
    "🔹 Imtihon rejimi: 20 ta test, 10
daqiqa, natija foiz va ballda\n"
    "🔹 Xatolarni takrorlash va tasodifiy
testlar\n"
    "🔹 Gistologiya seminar mashg'ulotlari:
mavzular bo'yicha vaziyatli testlar\n"
    "🔹 Anatomiya va Psixologiya yozma ish
biletlari\n"
```

```
    "🔹 Ilova ikki tilda ishlaydi: UZ | RU\n\n"
    "Vaqtingizni material qidirishga emas, tayyorgarlikka sarflang. 💪\n\n"
    "👉 Boshlash uchun pastdagi «Ilovani Ochish» tugmasini yoki ▦ menyuni bosing."
)

ABOUT_TEXT = (
    "ℹ️ Bot haqida\n\n"
    "TALABA MED tibbiyot talabalari uchun yaratilgan. Bot orqali ilovani ochib, "
    "o'zbek va rus potoklari uchun yakuniy fanlar testlari, imtihon rejimi, xatolarni takrorlash va yozma ish biletlaridan foydalanasiz.\n\n"
    "Ilova uchun uchala kanal/guruhga obuna bo'lish shart. Fikr va takliflaringizni "
    "«💬 Fikr bildirish» tugmasi orqali yuboring."
)

BTN_APP = "🌐 Ilovani ochish"
BTN_SCHEDULE = "🗓️ Dars jadvali"
BTN_ABOUT = "ℹ️ Bot haqida"
BTN_CHANNELS = "📢 Kanallar"
BTN_CONTACT = "📞 Aloqa"
BTN_FEEDBACK = "💬 Fikr bildirish"
BTN_ORALIQ = "📝 Oraliqlar"
BTN_BACK = "⬅️ Orqaga"

ORALIQ_PREFIX = "oraliq_"      # PDF fayl nomi shu so'z bilan boshlanishi kerak
FILE_PREFIX = "📄 "
```

```
MENU = ReplyKeyboardMarkup(
    keyboard=[
        [KeyboardButton(text=BTN_APP),
KeyboardButton(text=BTN_SCHEDULE)],
        [KeyboardButton(text=BTN_ABOUT),
KeyboardButton(text=BTN_CHANNELS)],
        [KeyboardButton(text=BTN_CONTACT),
KeyboardButton(text=BTN_FEEDBACK)],
        [KeyboardButton(text=BTN_ORALIQ)],
    ],
    resize_keyboard=True,
)

feedback_mode = {}   # user_id -> "anon" |
"named" | "choose"
last_feedback = {}   # user_id -> oxirgi
yuborilgan vaqt

CHANNELS = [
    ("🩺 Talaba Med", "@TalabaMed_2025"),
    ("📚 MedEdu", "@MedEdu_uz"),
    ("💬 Talaba Med dars (guruh)",
"@talabamed_dars"),
]

bot = Bot(TOKEN)
dp = Dispatcher()


async def not_subscribed(user_id):
    result = []
    for name, username in CHANNELS:
        try:
```

```
            m = await
bot.get_chat_member(username, user_id)
            if m.status in ("left",
"kicked"):
                result.append((name,
username))
            elif m.status == "restricted"
and not m.is_member:
                result.append((name,
username))
        except Exception as e:
            print("Xato:", username, e)
            result.append((name, username))
    return result


def sub_keyboard(channels):
    rows = [
        [InlineKeyboardButton(text=n,
url=f"https://t.me/{u[1:]}")]
        for n, u in channels
    ]

rows.append([InlineKeyboardButton(text="✅
Tekshirish", callback_data="check")])
    return
InlineKeyboardMarkup(inline_keyboard=rows)


def app_keyboard(url, text="🌐 Saytni
ochish"):
    return
InlineKeyboardMarkup(inline_keyboard=[[
        InlineKeyboardButton(text=text,
```

```
web_app=WebAppInfo(url=url))
    ]])


async def send_main(chat_id):
    if os.path.exists("welcome.jpg"):
        await bot.send_photo(chat_id,
FSInputFile("welcome.jpg"),
caption=WELCOME_TEXT, reply_markup=MENU)
    else:
        await bot.send_message(chat_id,
WELCOME_TEXT, reply_markup=MENU)

    try:
        await bot.set_chat_menu_button(
            chat_id=chat_id,
            menu_button=MenuButtonWebApp(
                text="Ilovani Ochish",

web_app=WebAppInfo(url=SITE_URL),
            ),
        )
    except Exception as e:
        print("Menu tugma xatosi:", e)


# ---------- Foydalanuvchilarni hisoblash
(Supabase) ----------

def db_headers(extra=None):
    h = {"apikey": SUPABASE_KEY, "Content-
Type": "application/json"}
    if SUPABASE_KEY.startswith("eyJ"):
        h["Authorization"] = f"Bearer
```

```
{SUPABASE_KEY}"
    if extra:
        h.update(extra)
    return h


async def save_user(user_id, first_name,
username, is_member=None):
    if not SUPABASE_URL or not
SUPABASE_KEY:
        return
    row = {
        "user_id": user_id,
        "first_name": first_name,
        "username": username,
        "last_seen":
datetime.now(timezone.utc).isoformat(),
    }
    if is_member is not None:
        row["is_member"] = is_member
        row["left_at"] = None if is_member
else datetime.now(timezone.utc).isoformat()
    try:
        async with aiohttp.ClientSession()
as s:
            async with s.post(
                f"
{SUPABASE_URL}/rest/v1/bot_users?
on_conflict=user_id",

headers=db_headers({"Prefer":
"resolution=merge-
duplicates,return=minimal"}),
                json=row,
```

```
timeout=aiohttp.ClientTimeout(total=10),
            ) as r:
                if r.status >= 300:
                    print("DB xato:",
r.status, await r.text())
    except Exception as e:
        print("DB xatosi:", e)


async def db_count(column=None, since=None,
extra=""):
    url = f"
{SUPABASE_URL}/rest/v1/bot_users?
select=user_id{extra}"
    if column and since:
        url += f"&{column}=gte." +
quote(since.isoformat(), safe="")
    async with aiohttp.ClientSession() as
s:
        async with s.get(
            url,
            headers=db_headers({"Prefer":
"count=exact", "Range": "0-0"}),

timeout=aiohttp.ClientTimeout(total=15),
        ) as r:
            return
int(r.headers.get("Content-Range",
"*/0").split("/")[-1])


# ---------- Obunadan chiqqanlarni aniqlash
----------
```

```
async def get_flag(user_id):
    """Bazadagi is_member qiymati: True /
False / None (noma'lum)."""
    if not SUPABASE_URL or not
SUPABASE_KEY:
        return None
    try:
        async with aiohttp.ClientSession()
as s:
            async with s.get(
                f"
{SUPABASE_URL}/rest/v1/bot_users?
select=is_member&user_id=eq.{user_id}",
                headers=db_headers(),

timeout=aiohttp.ClientTimeout(total=10),
            ) as r:
                data = await r.json()
                if data:
                    return
data[0].get("is_member")
    except Exception as e:
        print("get_flag xatosi:", e)
    return None


async def patch_user(user_id, fields):
    if not SUPABASE_URL or not
SUPABASE_KEY:
        return
    try:
        async with aiohttp.ClientSession()
as s:
```

```
            async with s.patch(
                f"
{SUPABASE_URL}/rest/v1/bot_users?
user_id=eq.{user_id}",

headers=db_headers({"Prefer":
"return=minimal"}),
                json=fields,

timeout=aiohttp.ClientTimeout(total=10),
            ) as r:
                if r.status >= 300:
                    print("DB patch xato:",
r.status, await r.text())
    except Exception as e:
        print("patch_user xatosi:", e)


async def warn_left(user_id, left):
    names = ", ".join(n for n, _ in left)
    try:
        await bot.send_message(
            user_id,
            "⚠️ Siz quyidagilardan chiqib
ketdingiz:\n" + names + "\n\n"
            "Shu sababli ilova va yangi
fayllardan foydalanish yopildi. "
            "Davom etish uchun qayta obuna
bo'ling va «Tekshirish» tugmasini bosing:",

reply_markup=sub_keyboard(left),
        )
    except Exception as e:
        print("Ogohlantirish yuborilmadi:",
```

```
user_id, e)


async def recheck_user(user_id, flag):
    """Obunani qayta tekshiradi, bazani
yangilaydi, chiqib ketgan bo'lsa
ogohlantiradi."""
    left = await not_subscribed(user_id)
    now = not left
    if flag is not now:
        await patch_user(user_id, {
            "is_member": now,
            "left_at": None if now else
datetime.now(timezone.utc).isoformat(),
        })
    if flag is True and not now:
        await warn_left(user_id, left)


@dp.chat_member()
async def on_member_change(ev:
ChatMemberUpdated):
    uname = ("@" + (ev.chat.username or
"")).lower()
    if uname not in [u.lower() for _, u in
CHANNELS]:
        return
    old, new = ev.old_chat_member.status,
ev.new_chat_member.status
    if new in ("left", "kicked") and old
not in ("left", "kicked"):
        uid = ev.new_chat_member.user.id
        flag = await get_flag(uid)
        await recheck_user(uid, flag)
```

```
async def sweep_once():
    """Bazadagi hammani aylanib chiqadi
(har birini ~0.2 soniyada)."""
    offset = 0
    while True:
        async with aiohttp.ClientSession()
as s:
            async with s.get(
                f"
{SUPABASE_URL}/rest/v1/bot_users?
select=user_id,is_member&order=user_id.asc&
limit=500&offset={offset}",
                headers=db_headers(),

timeout=aiohttp.ClientTimeout(total=30),
            ) as r:
                rows = await r.json()
        if not rows:
            break
        for row in rows:
            try:
                await
recheck_user(row["user_id"],
row.get("is_member"))
            except Exception as e:
                print("sweep xatosi:", e)
            await asyncio.sleep(0.2)
        offset += 500


async def sweep_loop():
    await asyncio.sleep(300)
```

```
    while True:
        if SUPABASE_URL and SUPABASE_KEY:
            try:
                await sweep_once()
            except Exception as e:
                print("sweep_once xatosi:",
e)
        await asyncio.sleep(6 * 3600)


@dp.message(Command("stats"))
async def stats(msg: Message):
    if msg.from_user.id != ADMIN_ID:
        return
    if not SUPABASE_URL or not
SUPABASE_KEY:
        await msg.answer("Baza ulanmagan
(SUPABASE_URL / SUPABASE_KEY yo'q).")
        return
    try:
        now = datetime.now(TZ)
        today = now.replace(hour=0,
minute=0, second=0, microsecond=0)
        week = today - timedelta(days=6)
        total = await db_count()
        new_today = await
db_count("first_seen", today)
        active_today = await
db_count("last_seen", today)
        new_week = await
db_count("first_seen", week)
        members = await
db_count(extra="&is_member=eq.true")
        left_n = await
```

```
db_count(extra="&is_member=eq.false")
        await msg.answer(
            "📊 Statistika\n\n"
            f"👥 Jami foydalanuvchi:
{total}\n"
            f"🆕 Bugun yangi:
{new_today}\n"
            f"🔥 Bugun faol:
{active_today}\n"
            f"📅 Oxirgi 7 kunda yangi:
{new_week}\n\n"
            f"✅ Hozir obunada:
{members}\n"
            f"🚪 Obunasiz (chiqib ketgan
yoki hali obuna bo'lmagan): {left_n}"
        )
    except Exception as e:
        print("stats xatosi:", e)
        await msg.answer("Statistikani
olishda xatolik. Loglarni tekshiring.")


@dp.message(CommandStart())
async def start(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    left = await
not_subscribed(msg.from_user.id)
    await save_user(msg.from_user.id,
msg.from_user.first_name,
msg.from_user.username, not left)
    if left:
        await msg.answer(
            "Botdan foydalanish uchun
```

```
quyidagilarga obuna bo'ling, "
            "so'ng «Tekshirish» tugmasini
bosing:",

reply_markup=sub_keyboard(left),
        )
    else:
        await send_main(msg.chat.id)


@dp.message(Command("menu"))
async def menu_cmd(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    await msg.answer("📋 Menyu pastda.
Yopish uchun telefondagi orqaga tugmasini
yoki ▦ ni bosing.", reply_markup=MENU)


@dp.callback_query(F.data == "check")
async def check(cb: CallbackQuery):
    left = await
not_subscribed(cb.from_user.id)
    if left:
        await cb.answer("Hali hammasiga
obuna bo'lmadingiz!", show_alert=True)
        try:
            await
cb.message.edit_reply_markup(reply_markup=s
ub_keyboard(left))
        except Exception:
            pass
    else:
        await cb.answer()
```

```
        await save_user(cb.from_user.id,
cb.from_user.first_name,
cb.from_user.username, True)
        try:
            await cb.message.delete()
        except Exception:
            pass
        await send_main(cb.message.chat.id)


# ---------- Pastdagi tugmalar paneli -----
-----

async def need_subscribe(msg: Message):
    """Obuna bo'lmagan bo'lsa, so'rov
yuboradi va True qaytaradi."""
    left = await
not_subscribed(msg.from_user.id)
    if left:
        await msg.answer(
            "Avval quyidagilarga obuna
bo'ling, so'ng «Tekshirish» tugmasini
bosing:",

reply_markup=sub_keyboard(left),
        )
        return True
    return False


@dp.message(F.text == BTN_APP)
async def btn_app(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
```

```
    if await need_subscribe(msg):
        return
    await msg.answer("Ilovani ochish uchun
tugmani bosing.",
reply_markup=app_keyboard(SITE_URL))


@dp.message(F.text == BTN_SCHEDULE)
async def btn_schedule(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    if await need_subscribe(msg):
        return
    await msg.answer("🗓 Dars jadvalini
ochish uchun tugmani bosing.",

reply_markup=app_keyboard(SCHEDULE_URL, "🗓
Dars jadvalini ochish"))


@dp.message(F.text == BTN_ABOUT)
async def btn_about(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    await msg.answer(ABOUT_TEXT)


@dp.message(F.text == BTN_CHANNELS)
async def btn_channels(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    rows = [[InlineKeyboardButton(text=n,
url=f"https://t.me/{u[1:]}")] for n, u in
CHANNELS]
```

```
    await msg.answer("🎣 Bizning kanal va
guruhlar:",
reply_markup=InlineKeyboardMarkup(inline_ke
yboard=rows))


@dp.message(F.text == BTN_CONTACT)
async def btn_contact(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    kb =
InlineKeyboardMarkup(inline_keyboard=[[
        InlineKeyboardButton(text="✉️
Yozish", url=f"https://t.me/{CONTACT[1:]}")
    ]])
    await msg.answer(f"📞 Aloqa uchun:
{CONTACT}", reply_markup=kb)


@dp.message(F.text == BTN_FEEDBACK)
async def btn_feedback(msg: Message):
    feedback_mode[msg.from_user.id] =
"choose"
    kb =
InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🕶️
Anonim yuborish",
callback_data="fb_anon")],
        [InlineKeyboardButton(text="👤
Ismim bilan yuborish",
callback_data="fb_named")],
        [InlineKeyboardButton(text="❌
Bekor qilish", callback_data="fb_cancel")],
    ])
```

```
    await msg.answer(
        "💬 Bot haqida fikringizni
bildiring.\n\n"
        "Anonim yuborsangiz, sizning
ismingiz va akkauntingiz egasiga
ko'rsatilmaydi. "
        "Qanday yuborasiz?",
        reply_markup=kb,
    )


# ---------- Oraliqlar (PDF fayllar) ------
----

file_cache = {}   # fayl nomi -> Telegram
file_id (qayta yuklamaslik uchun)


def oraliq_files():
    """Repodagi oraliq_*.pdf fayllarini
topadi: {tugma nomi: fayl yo'li}"""
    base =
os.path.dirname(os.path.abspath(__file__))
    result = {}
    try:
        names = sorted(os.listdir(base))
    except Exception:
        return result
    for fn in names:
        if
fn.lower().startswith(ORALIQ_PREFIX) and
fn.lower().endswith(".pdf"):
            title =
fn[len(ORALIQ_PREFIX):-4].replace("_", "
```

```
").strip()
            if title:
                result[FILE_PREFIX + title]
= os.path.join(base, fn)
    return result


@dp.message(F.text == BTN_ORALIQ)
async def btn_oraliq(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    files = oraliq_files()
    if not files:
        await msg.answer("📝 Oraliq
fayllari hozircha yuklanmagan. Tez orada
qo'shiladi!")
        return
    rows = [[KeyboardButton(text=t)] for t
in files]

rows.append([KeyboardButton(text=BTN_BACK)]
)
    kb = ReplyKeyboardMarkup(keyboard=rows,
resize_keyboard=True)
    await msg.answer("📝 Oraliqlar. Fanni
tanlang:", reply_markup=kb)


@dp.message(F.text == BTN_BACK)
async def btn_back(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    await msg.answer("📋 Asosiy menyu",
reply_markup=MENU)
```

```
@dp.message(F.text.startswith(FILE_PREFIX))
async def btn_oraliq_file(msg: Message):
    feedback_mode.pop(msg.from_user.id,
None)
    if await need_subscribe(msg):
        return
    path = oraliq_files().get(msg.text)
    if not path:
        await msg.answer("Bu fayl
topilmadi. /menu ni bosib qayta urinib
ko'ring.")
        return
    caption = "📝 " +
msg.text[len(FILE_PREFIX):] + "
(oraliq)\n\nTALABA MED"
    try:
        fid = file_cache.get(path)
        sent = await
msg.answer_document(fid or
FSInputFile(path), caption=caption,
reply_markup=MENU, protect_content=True)
        if not fid and sent.document:
            file_cache[path] =
sent.document.file_id
    except Exception as e:
        print("PDF yuborishda xato:", e)
        await msg.answer("Faylni yuborib
bo'lmadi. Keyinroq urinib ko'ring.")


@dp.callback_query(F.data.in_({"fb_anon",
"fb_named", "fb_cancel"}))
```

```
async def fb_choice(cb: CallbackQuery):
    uid = cb.from_user.id
    if cb.data == "fb_cancel":
        feedback_mode.pop(uid, None)
        await cb.answer("Bekor qilindi")
        try:
            await cb.message.delete()
        except Exception:
            pass
        return
    feedback_mode[uid] = "anon" if cb.data
== "fb_anon" else "named"
    await cb.answer()
    how = "anonim" if cb.data == "fb_anon"
else "ismingiz bilan"
    try:
        await cb.message.edit_text(f"✍
Fikringizni bitta xabar qilib yozing ({how}
yuboriladi).\nBekor qilish uchun pastdagi
tugmalardan birini bosing.")
    except Exception:
        pass


@dp.message(F.text &
~F.text.startswith("/"))
async def feedback_text(msg: Message):
    uid = msg.from_user.id
    mode = feedback_mode.get(uid)
    if mode not in ("anon", "named"):
        return
    if not ADMIN_ID:
        feedback_mode.pop(uid, None)
        await msg.answer("Fikr qabul qilish
```

```
hozircha ishlamayapti.")
        return
    now = time.time()
    if now - last_feedback.get(uid, 0) <
30:
        await msg.answer("Iltimos, biroz
kutib, keyin yuboring.")
        return
    text = msg.text.strip()[:1500]
    if mode == "anon":
        head = "💬 Yangi fikr (anonim)"
    else:
        u = msg.from_user
        name = " ".join(x for x in
[u.first_name, u.last_name] if x) or "Ism
yo'q"
        uname = f" @{u.username}" if
u.username else ""
        head = f"💬 Yangi fikr\n👤 {name}
{uname} (ID: {u.id})"
    try:
        await bot.send_message(ADMIN_ID, f"
{head}\n\n{text}")
    except Exception as e:
        print("Fikrni yuborishda xato:", e)
        await msg.answer("Yuborib bo'lmadi,
keyinroq urinib ko'ring.")
        return
    last_feedback[uid] = now
    feedback_mode.pop(uid, None)
    await msg.answer("✅ Rahmat! Fikringiz
yuborildi.")
```

```
# ---------- Sayt uchun obuna tekshiruvi --
--------

def user_from_init_data(init_data):
    """Telegram initData imzosini
tekshiradi va foydalanuvchini qaytaradi."""
    try:
        pairs = dict(parse_qsl(init_data,
keep_blank_values=True))
        received = pairs.pop("hash", None)
        if not received:
            return None
        data_check = "\n".join(f"{k}={v}"
for k, v in sorted(pairs.items()))
        secret = hmac.new(b"WebAppData",
TOKEN.encode(), hashlib.sha256).digest()
        calc = hmac.new(secret,
data_check.encode(),
hashlib.sha256).hexdigest()
        if not hmac.compare_digest(calc,
received):
            return None
        if time.time() -
int(pairs.get("auth_date", "0")) > 86400:
            return None
        return json.loads(pairs["user"])
    except Exception as e:
        print("initData xatosi:", e)
        return None


@web.middleware
async def cors_mw(request, handler):
    if request.method == "OPTIONS":
```

```
        resp = web.Response()
    else:
        resp = await handler(request)
    resp.headers["Access-Control-Allow-
Origin"] = SITE_ORIGIN
    resp.headers["Access-Control-Allow-
Methods"] = "POST, OPTIONS"
    resp.headers["Access-Control-Allow-
Headers"] = "Content-Type"
    return resp


async def api_check(request):
    if request.method == "OPTIONS":
        return web.Response()
    try:
        body = await request.json()
    except Exception:
        return web.json_response({"ok":
False, "error": "bad_request"}, status=400)
    user =
user_from_init_data(body.get("initData",
""))
    if not user:
        return web.json_response({"ok":
False, "error": "invalid"}, status=403)
    left = await not_subscribed(user["id"])
    await save_user(user["id"],
user.get("first_name"),
user.get("username"), not left)
    missing = [{"name": n, "url":
f"https://t.me/{u[1:]}"} for n, u in left]
    return web.json_response({"ok": not
left, "missing": missing})
```

```
async def health(request):
    return web.Response(text="OK")


async def main():
    app = web.Application(middlewares=
[cors_mw])
    app.router.add_get("/", health)
    app.router.add_route("*", "/api/check",
api_check)
    runner = web.AppRunner(app)
    await runner.setup()
    await web.TCPSite(runner, "0.0.0.0",
int(os.environ.get("PORT", 8080))).start()
    asyncio.create_task(sweep_loop())
    await dp.start_polling(bot,
allowed_updates=dp.resolve_used_update_type
s())


asyncio.run(main())
```